

Integración de Servicios Cloud GCP

Ciclo de vida de un servicio, servidores reales, firewall, DNS y permisos

Semana 04 - Sesión 08

Rel Guzman



**Universidad
Tecnológica
del Perú**

- ▶ ¿Qué tres datos definen un socket?
- ▶ ¿Qué pasos hace el servidor que el cliente no necesita hacer, y por qué?
- ▶ En la práctica anterior, ¿qué comando les dijo qué proceso ocupaba el puerto 8080?

Logro de la sesión

Al finalizar, podrás describir el ciclo de vida de un servicio y qué rol se encarga de cada etapa, usar servidores y servicios reales (nginx, MySQL y el firewall del sistema), reconocer los puertos clásicos, describir el papel de un servidor DNS y administrar los permisos de los archivos de un proyecto.

- ▶ ¿Para qué sirve `systemctl` y qué diferencia hay entre `start` y `enable`?
- ▶ ¿Quién instala y quién mantiene los servidores donde corre una aplicación?
- ▶ ¿Han visto alguna vez un servicio caído en producción? ¿Quién lo levantó?

¿Para qué te servirá lo aprendido?

Un servicio no se instala y se olvida: hay que configurarlo, arrancarlo, vigilarlo, actualizarlo y darlo de baja. Saber quién responde por cada etapa te permite ubicarte en un equipo real y entender qué te delega un proveedor cloud y qué sigue siendo tuyo.

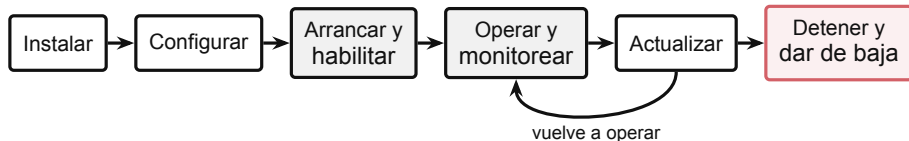
También podrás abrir y cerrar puertos con criterio, entender cómo el DNS apunta un dominio a un servidor y proteger archivos sensibles como el `.env` de un proyecto.

Ciclo de vida de un servicio y roles a cargo

Un servicio recorre siempre las mismas etapas

Idea central

El servidor que escribimos la sesión pasada moría al cerrar la terminal. Un servicio de verdad tiene una vida más larga: alguien lo instala, lo configura, lo arranca, lo vigila, lo actualiza y algún día lo apaga. Cada etapa deja rastro y tiene un responsable.



La etapa más larga es *operar*

Instalar toma minutos; operar dura años. Ahí ocurren las caídas y los registros que crecen.

Dar de baja también es trabajo

Un servicio olvidado sigue escuchando en un puerto y ya nadie lo actualiza: se vuelve la puerta de entrada más fácil.

Cada etapa tiene su comando

En Ubuntu, `systemd` es quien mantiene vivos los servicios. Lo vemos con `nginx`, un servidor web. Son los mismos comandos para cualquier servicio: solo cambia el nombre.

Lo esencial: instalar, encender, mirar

```
$ sudo apt install nginx
$ sudo systemctl start nginx
$ systemctl status nginx
$ sudo systemctl stop nginx
```

Con estos cuatro ya puedes levantar un servicio y comprobar si está vivo.

start frente a enable

`start` lo levanta ahora; `enable` hace que vuelva solo tras reiniciar la máquina.

Después: dejarlo fijo y mantenerlo

```
$ sudo systemctl enable nginx
$ sudo systemctl reload nginx
$ journalctl -u nginx
$ sudo systemctl disable --now nginx
```

reload frente a restart

`reload` relee la configuración sin cortar el servicio; `restart` lo apaga y lo vuelve a abrir.

Quién se encarga de cada etapa

En un equipo pequeño una sola persona hace todo; en uno grande, no

Los nombres cambian entre empresas, pero las responsabilidades son estas.

Desarrollador

Escribe el servicio y define qué necesita para arrancar: puerto, variables de entorno, dependencias.

Administrador de sistemas

(SysAdmin)

Instala, configura y mantiene el servidor: paquetes, usuarios, permisos, respaldos.

DevOps / SRE

Automatiza el despliegue y vigila que el servicio siga en pie: monitoreo, alertas e incidentes.

Cloud engineer

Construye y opera la infraestructura en la nube: VMs, redes, reglas de firewall, permisos de acceso y costos.

Arquitecto cloud

Decide el diseño: si el servicio corre en una VM propia o en un servicio administrado del proveedor.

Administrador de base de datos (DBA)

Responde por MySQL o PostgreSQL: esquema, respaldos, restauración y rendimiento de las consultas.

Analista de seguridad

Decide qué puertos se abren y desde dónde, revisa registros y exige que lo expuesto esté justificado.

Base de datos y firewall simulados

El mismo esqueleto, otra respuesta

```
TABLA = [(1, 'Ana', 'ana@utp.edu.pe'), ... ]

s.bind(('127.0.0.1', 3307))
s.listen(5)
while True:
    cli, dir = s.accept()
    consulta = cli.recv(1024).decode()
    cli.sendall(responder(consulta).encode())
    cli.close()
```

Qué cambia frente al de HTTP

Solo el puerto y lo que se devuelve: en vez de una página, filas de una tabla.

Puerto 3307

Se usa 3307 para no chocar con un MySQL real, que ocupa el 3306.

Conecta, consulta y lee

```
c.connect(('127.0.0.1', 3307))
c.sendall('SELECT * FROM usuarios'.encode())
respuesta = c.recv(4096)
c.close()
print(respuesta.decode())
```

Resultado

```
$ python3 cliente_mysql.py
Consulta enviada:
SELECT * FROM usuarios
1 | Ana | ana@utp.edu.pe
2 | Luis | luis@utp.edu.pe
3 | Sara | sara@utp.edu.pe
```

La misma secuencia que el cliente HTTP

`connect()`, `sendall()`, `recv()`. Cambia el puerto y el texto que se envía; el mecanismo es idéntico. Un MySQL real usa un formato binario más elaborado, pero por debajo también es esto.

Cliente

Quien inicia la conexión y pide algo. Termina cuando obtiene su respuesta.

Servidor

Quien escucha en un puerto y responde. Necesita estar activo antes que el cliente.

Servicio

Cualquier programa que el sistema mantiene en segundo plano para cumplir una función.

No todo servicio es un servidor

Un servidor siempre escucha peticiones; un servicio no tiene por qué hacerlo. `cron` ejecuta tareas a cierta hora, un servicio de respaldo copia archivos de madrugada y `systemd-timesyncd` ajusta el reloj: ninguno abre un puerto ni espera clientes, y aun así los tres son servicios.

El firewall es un servicio, no un servidor

Nadie se conecta al firewall. Se ejecuta en segundo plano revisando el tráfico que entra y decidiendo si dejarlo pasar. Por eso no tiene cliente ni responde peticiones: solo filtra.

Cómo razona

Recorre una lista de **reglas** en orden. Si alguna coincide con el origen y el puerto, aplica su acción. Si ninguna coincide, aplica la **política por defecto**.

Política por defecto: denegar

Se cierra todo y se abre solo lo necesario. Es el criterio recomendado en servidores.

Qué mira cada regla

Dirección de origen, puerto de destino, protocolo y la acción: permitir o denegar.

Escuchar no es ser accesible

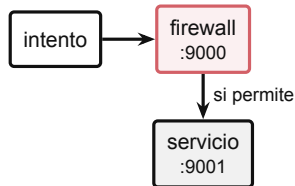
El programa puede estar activo y aun así el firewall impedir que lleguen las conexiones.

La política: el baneo va primero y sin puerto

```
POLITICA_POR_DEFEECTO = 'DENEGAR'
REGLAS = [
    {'origen': '203.0.113.66/32',
     'puerto': 'cualquiera', 'accion': 'DENEGAR'},
    {'origen': '192.168.100.0/24',
     'puerto': 22, 'accion': 'PERMITIR'},
    {'origen': '0.0.0.0/0',
     'puerto': 80, 'accion': 'PERMITIR'},
    ...
]
```

Es la puerta, no el destino

Se queda corriendo y todo pasa por él. Lo que permite lo **reenvía** al servicio; lo que deniega no llega nunca.



Por qué la regla 1 va primero

Es la única sin puerto: cierra esa IP en **toda** la máquina. Si estuviera al final nunca se aplicaría al 80, porque la regla 3 ya habría dejado pasar. Es el mismo motivo por el que en `ufw` un baneo se agrega con `insert 1` y no al final.

El servicio está activo; el firewall decide quién llega

Terminal 2: el firewall, corriendo

```
$ python3 servicio_firewall.py
FIREWALL ACTIVO en 127.0.0.1:9000
Esperando intentos...

192.168.100.161 -> 22 PERMITIR (regla 2)
203.0.113.9 -> 3306 DENEGAR (por defecto)
203.0.113.66 -> 80 DENEGAR (regla 1)
```

Terminal 1: el servicio, corriendo

```
$ python3 servicio.py
Servicio ACTIVO en 127.0.0.1:9001

[15:33]atendiendo a 192.168.100.161
```

Terminal 3: los intentos

```
$ python3 intento_1.py
PERMITIDO. Servicio atendido

$ python3 intento_2.py
DENEGADO (política por defecto)

$ python3 intento_3.py
DENEGADO (regla 1)
```

Mira la terminal 1

Solo registró **un** intento. Los otros dos nunca llegaron al servicio: murieron en el firewall.

Escuchar no es ser accesible

El servicio nunca se detuvo. Lo único que cambió fue quién tenía permiso de llegar. Por eso, ante un servicio que "no responde", se revisan dos cosas distintas: si hay alguien escuchando (`ss -ltnp`) y si se le permite llegar (`ufw status`).

Servidores y servicios reales en uso

Instala y comprueba

```
$ sudo apt update
$ sudo apt install nginx -y
$ systemctl status nginx
$ curl -I 127.0.0.1
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
```

Es el mismo servicio que usamos en el ciclo de vida.

Qué aporta frente al nuestro

Atiende muchos clientes a la vez, sirve archivos reales, registra accesos y admite HTTPS.

Dónde viven los archivos

```
/var/www/html/index.html
```

Lo que dejes ahí queda publicado.

Instala, activa y conéctate

```
$ sudo apt install mysql-server -y
$ sudo systemctl start mysql
$ sudo systemctl enable mysql
$ systemctl is-active mysql
active
$ sudo mysql -e 'SHOW DATABASES;'
```

Lo que simulamos, en real

Aquí también hay un servidor escuchando y un cliente (`mysql`) que envía consultas y recibe filas.

Comprueba el nombre real

Con MariaDB la unidad cambia de nombre. Búscala con `systemctl list-unit-files '*mysql*'.`

Política por defecto y permisos

```
$ sudo ufw default deny incoming
$ sudo ufw allow 22/tcp
$ sudo ufw allow 80/tcp
$ sudo ufw enable
$ sudo ufw status numbered
```

Abre el 22 antes de activarlo: si administras la máquina por SSH y no lo permites, pierdes el acceso.

Es el mismo razonamiento

default deny es la política por defecto de servicio_firewall.py; cada allow es una regla de la lista, y un baneo por IP se agrega con insert 1.

Regla por origen

```
$ sudo ufw allow from \
  192.168.1.0/24 to any \
  port 3306
```

Cerrar la base de datos al exterior

La app se conecta desde la propia máquina o desde la red interna; el 3306 nunca debe estar abierto a Internet.

Publicar solo lo necesario

En un servidor web se dejan abiertos 80 y 443, y se cierra todo lo demás.

Limitar la administración

Permitir el 22 solo desde la IP de la oficina o de la VPN, no desde cualquier origen.

Bloquear un origen abusivo

Banear la IP que está intentando adivinar contraseñas o saturando el servidor.

Cerrar puertos de desarrollo

3000, 8000 y paneles internos sirven mientras desarrollas; en producción no deben quedar expuestos.

Separar ambientes

Que el servidor de pruebas solo acepte conexiones del equipo de desarrollo.

1. Publica un sitio estático en la VM

```
$ echo '<h1>Sitio de prueba</h1>' \  
| sudo tee /var/www/html/index.html  
$ sudo ufw allow 80/tcp  
$ sudo ufw enable
```

Desde el host, abre `http://VM_IP` en el navegador: debe verse la página.

2. Banea la IP del host

```
$ sudo ufw insert 1 deny from HOST_IP
```

3. Comprueba y revierte

```
$ sudo ufw status numbered  
[1]Anywhere DENY IN HOST_IP  
[2]80/tcp ALLOW IN Anywhere  
  
$ sudo ufw delete 1
```

Recarga el navegador en cada paso y anota qué ocurre.

deny frente a reject

deny descarta el paquete en silencio: el navegador se queda cargando hasta agotar el tiempo. reject avisa y el error es inmediato.

Servicios y puertos clásicos

Por qué importan

El puerto indica a cuál de esos servicios llega cada conexión. Los números que siguen son **estándares**: la convención que todo el mundo espera. Pero, como ya vieron, nada impide levantar un servidor en el puerto que quieran —lo hicimos en el 8080 y en el 3307—; solo que entonces el cliente debe indicarlo.

Acceso y web

- 22 SSH: administración remota.
- 80 HTTP: web sin cifrar.
- 443 HTTPS: web cifrada.
- 53 DNS: resolución de nombres.

Datos y aplicaciones

- 3306 MySQL.
- 5432 PostgreSQL.
- 8000 y 3000: aplicaciones Node o Python en desarrollo.
- 25 y 587 SMTP: correo saliente.

Qué hace un analista de seguridad ante un puerto abierto

Cada puerto abierto es superficie de ataque

El trabajo no es cerrarlo todo, sino justificar uno por uno: qué servicio escucha ahí, quién necesita alcanzarlo y desde dónde.

1. Inventariar

```
$ sudo ss -ltnp
```

Qué escucha, en qué dirección y qué programa lo abrió.

2. Mirar desde fuera

```
$ nmap -sV VM_IP
```

Ve lo mismo que vería un atacante, incluida la versión del servicio.

3. Preguntar por cada puerto

¿Debe ser accesible desde Internet, solo desde la red interna o solo desde la propia máquina?

4. Reducir y vigilar

Cerrar lo que no se justifica, restringir por origen, mantener el servicio actualizado y revisar los registros de acceso.

El servidor DNS

Qué problema resuelve

Las personas recordamos `google.com`; las máquinas necesitan una IP. El DNS es el servicio que convierte lo primero en lo segundo, y escucha en el puerto 53. Si falla, el navegador no encuentra el sitio aunque el servidor esté funcionando.

Registros frecuentes

A: apunta un nombre a una IPv4.

AAAA: lo mismo, con IPv6.

CNAME: alias de otro nombre.

MX: a qué servidor va el correo.

TXT: notas y verificaciones de dominio.

Lo que harás en la práctica

Al contratar un hosting o crear una VM te entregan una **IP pública**. Para que tu dominio apunte ahí, entras al panel del proveedor del dominio y editas el **registro A**, poniendo esa IP.

Y el `www`

Suele resolverse con un CNAME de `www.tudominio.com` hacia `tudominio.com`, para no repetir la IP en dos sitios.

Gestión de permisos

Idea central

Ubuntu protege cada archivo indicando qué acciones puede realizar el **propietario**, el **grupo** y el **resto de usuarios**. Un servidor lo usan varias personas y varios servicios: los permisos evitan que un programa cualquiera lea las credenciales de otro.

¿Quién?

u: propietario.

g: grupo.

o: otros usuarios.

¿Qué puede hacer?

r: leer.

w: modificar.

x: ejecutar o atravesar un directorio.

Los archivos del proyecto (carpeta Sesion08/proyecto)

```
rgap@ubuntu:~/proyecto$ ls -la
-rw-rw-r-- 1 rgap rgap 79 Sep 1 13:51 .env
-rw-rw-r-- 1 rgap rgap 2845 Sep 1 13:50 servidor_http.py
```

Con `ls -a` también aparecen los archivos ocultos, los que empiezan con punto.

-	rw-	rw-	r--
Archivo regular. Una <code>a</code> indicaría directorio.	El propietario lee y modifica; todavía no ejecuta.	El grupo <code>rgap</code> lee y modifica.	Los demás solo leen. Aquí está el problema del <code>.env</code> .

Caso 1: un .py necesita intérprete y permiso x

Primera línea del script

```
#!/usr/bin/env python3
from http.server import \
    BaseHTTPRequestHandler, HTTPServer
...
```

Le indica a Ubuntu qué programa debe interpretarlo.

Por qué python3 archivo.py sí funciona

Ahí ejecutas Python, y Python solo lee el archivo.
./archivo.py exige que el archivo mismo sea ejecutable.

Antes y después

```
$ ./servidor_http.py
bash: Permiso denegado
$ chmod u+x servidor_http.py
$ ./servidor_http.py
Escuchando en
http://127.0.0.1:8080
```

Caso 2: el `.env` guarda secretos y debe cerrarse

Qué contiene

```
DB_USER=app_web  
DB_PASSWORD=cambia-esta-clave  
API_KEY=clave-de-ejemplo-no-real
```

Credenciales que dan acceso real a la base de datos y a servicios de pago.

Ciérralo al propietario

```
$ chmod 600 .env  
$ ls -l .env  
-rw----- 1 rgap rgap 223 .env
```

600: el propietario lee y escribe; nadie más entra.

El permiso no reemplaza al `.gitignore`

`chmod` protege el archivo en esta máquina. Para que no llegue al repositorio, además debe estar listado en `.gitignore`.

Simbólica: dices qué cambiar

```
$ chmod u+x app.py  
$ chmod o-r .env
```

La primera agrega ejecución al propietario. La segunda quita la lectura a los demás.

Los tres que verás casi siempre

```
chmod 600 .env rw- --- --- solo el propietario  
chmod 644 index.html rw- r-- r-- todos leen, solo tú editas  
chmod 777 archivo rwx rwx rwx todos hacen todo: evítalo
```

777 no arregla la causa de un error de permisos; solo amplía quién puede provocarlo.

Numérica: $r=4$, $w=2$, $x=1$

```
4+2+1 = 7  rwx  
4+2 = 6  rw-  
4 = 4  r--
```

Un dígito por cada grupo: propietario, grupo y otros.

1. Crea sitio.py y Pruébalo

```
from http.server import \
    BaseHTTPRequestHandler, HTTPServer

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header('Content-Type',
            'text/html; charset=utf-8')
        self.end_headers()
        self.wfile.write(HTML.encode())

HTTPServer(("127.0.0.1", 3000),
    Handler).serve_forever()
```

El HTML va en la constante HTML, arriba del todo.

2. Ponlo detrás de nginx

```
$ sudo nano \
/etc/nginx/sites-available/default

location / {
    proxy_pass http://127.0.0.1:3000;
}

$ sudo nginx -t
$ sudo systemctl reload nginx
```

3. Corrige los permisos

```
$ chmod u+x iniciar.sh
$ chmod 600 .env
```

Comprobación

Desde el host, `http://VM_IP` muestra la página con el 3000 cerrado en `ufw`: quien atiende afuera es `nginx`. Explica por qué el 3000 no necesita abrirse.

A vibrant watercolor splash background in shades of blue, purple, and pink, with the word "gracias" written in a white, cursive script across the center.

gracias



Universidad
Tecnológica
del Perú

- ▶ systemd: páginas de manual de `systemctl`, `journalctl`, `systemd.unit` y `systemd.service`, freedesktop.org/software/systemd/man.
- ▶ Documentación de Ubuntu Server: nginx, MySQL, UFW y systemd-resolved, documentation.ubuntu.com/server.
- ▶ Documentación oficial de nginx, nginx.org/en/docs y docs.nginx.com.
- ▶ Páginas de manual de `ufw`, `ss` y `nmap`.
- ▶ Documentación de Python: módulo `socket`, docs.python.org/3/library/socket.html.
- ▶ RFC 1034 y RFC 1035: conceptos y especificación del sistema de nombres de dominio.
- ▶ GNU Coreutils: manual de `chmod`; páginas de manual de `ls`, `chmod` y `env`.
- ▶ Documentación de Python: módulo `http.server`, docs.python.org/3/library/http.server.html.
- ▶ nginx: *Beginner's Guide* y `ngx_http_proxy_module`, nginx.org/en/docs.
- ▶ Documentación oficial de Google Antigravity CLI, antigravity.google/docs/cli.
- ▶ Material base proporcionado: *Fundamentos de Linux — Programa completo*; identidad visual y estructura docente de la Sesión 05.